

Allowed Non-Determinism in the Coordination Knowledge Substrate Pattern: Standalone Treatment of the Five Categories the Determinism Contract Permits

A derivation note from the Coordination Knowledge Substrate (CKS) pattern.

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: 5 May 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the five categories of allowed non-determinism enumerated under the determinism contract — LLM token output, cell execution timing, inter-cell communication timing, external-system responses, and hardware/infrastructure variation — as a standalone architectural specification with each category paired to its bounding mechanism, distinct from the five substrate-side guarantees treated in companion notes.

Abstract

The Coordination Knowledge Substrate (CKS) pattern's determinism contract has two faces. One side specifies the determinism the architecture *requires* — five guarantees over substrate state, cell behavior, change addressability, conflict preservation, and source-of-truth precedence. The other side specifies the non-determinism the architecture *permits* — five categories where current technology cannot deliver determinism and the architecture is designed to operate with that fact. Companion notes formalize the five guarantees individually. This note formalizes the five permitted-non-determinism categories as a standalone specification. Each category has independent architectural content: a precise statement of what the category includes, why it is operationally necessary, and which architectural mechanism bounds it so that non-determinism in the category does not propagate to substrate. The five categories are not a residual list; they are a positive specification of where non-determinism may live and how each location is fenced. The note states the five categories with their bounding mechanisms, distinguishes allowed non-determinism from four adjacent patterns commonly conflated with it, names the failure modes that misattribute non-determinism between allowed and forbidden, and provides an operational test for whether a system's non-determinism architecture satisfies the boundaries.

1. Why allowed non-determinism needs to be formalized as standalone

The CKS pattern's determinism contract (A1.10, derived from §4.1, §6.2, §11.3 of the source paper) is a paired commitment: the architecture commits to specific determinism guarantees over substrate behavior, and it commits to specific categories of non-determinism that are permitted at layers the

guarantees do not constrain. The integrating note A2.55 enumerated both halves at the integrating level. Companion notes A2.57–A2.61 formalize each of the five substrate-side guarantees as a standalone commitment.

The remaining half — the allowed-non-determinism specification — also needs standalone treatment, for three reasons.

First, the categories are *specifications*, not concessions. Each category names a layer where current technology produces non-deterministic behavior; each is paired with an architectural mechanism that bounds that non-determinism so it does not reach substrate. Read as concessions ("the contract is weak in these places"), the categories invite implementations that treat them as license to permit non-determinism freely. Read as specifications ("non-determinism is permitted here, bounded thus, prevented from propagating thus"), the categories are precise architectural content.

Second, the typical implementation failure pattern is misattribution. Implementations under operational pressure routinely either over-claim determinism (treating all non-determinism as forbidden, producing architectures that cannot operate with current LLM technology at all) or under-claim (allowing non-determinism in categories that violate the guarantees, producing architectures whose substrate guarantees fail in deployment). The line between allowed and forbidden is precise but operationally subtle, and standalone formalization is what makes the line legible.

Third, the allowed-non-determinism specification is consequential prior art. Derivations targeting AI-coordination architectures with bounded non-determinism, mixed-determinism architectures, or non-determinism-aware AI infrastructure are more defensibly contested when the five categories, paired to their bounding mechanisms, are publicly formalized as a standalone artifact.

2. Category (a): LLM token output non-determinism

What it is. LLM token outputs vary across executions even with identical prompts. The variation arises from sampling configurations (temperature greater than zero, top-k or top-p sampling), floating-point and parallel-computation effects in the underlying inference stack, model-internal state differences across executions, and other model-internal sources. The same prompt may produce different token sequences; the same input distribution may produce different output distributions; reasoning paths internal to the model may diverge between calls.

Why it is operationally necessary. Current LLM technology does not reliably produce bit-identical outputs across executions. Even with sampling configured for determinism (temperature zero, fixed seeds where supported), production inference stacks introduce variation through floating-point precision differences, GPU computation ordering, and version drift in model serving. The CKS architecture is designed to operate with current LLM technology as it exists; requiring LLM-output determinism would constrain deployments to specific configurations that are often unavailable and that do not, in any case, guarantee bit-identity in practice.

How it is bounded. Property B of the AI-as-substrate-mediator commitment (A2.20, derived from §4.2) requires that LLM writes to substrate occur under orchestration rules — the rule structures what gets written, and the writes are rule-conformant per Guarantee B (A2.58). Property C (A2.21) requires that the LLM not hold substrate-relevant state outside substrate. Together, Properties B and C bound LLM non-determinism to

the cell-execution boundary: specific token content may vary across executions, but the *structure* of what reaches substrate is rule-determined, and non-determinism does not accumulate across executions in ways that propagate to substrate. Substrate's determinism per Guarantees A, C, D, and E is preserved.

3. Category (b): Cell execution timing non-determinism

What it is. Cells execute at varying times. The timing of a cell's execution depends on triggering events, infrastructure scheduling, resource availability, and queue depths in the host environment. The same triggering condition may cause a cell to execute at different times across deployments and across runs within a single deployment; cell execution latency varies; the moment a cell's commit becomes substrate state varies.

Why it is operationally necessary. Deployments cannot synchronize all cell executions because cells run on distributed infrastructure with operational variation. Requiring synchronized cell timing would constrain the architecture to specific (often unavailable) infrastructure configurations and would foreclose practical scaling.

How it is bounded. Boundary crossings between cell and substrate are atomic per A2.10 (derived from §4.1): each cell-write is a single substrate operation in which content and provenance are committed together. Atomicity is what makes cell-execution timing irrelevant to substrate determinism. At any given moment, substrate state is the result of all completed atomic commits, regardless of when each commit occurred. Guarantee A (A2.57) holds because reads return the content of substrate's current state, not state-as-of-some-particular-cell-execution-time. Guarantee C (A2.59) holds because each change is atomically committed; the moment the change becomes substrate content is the moment of commit, and the change is addressable from that moment onward.

4. Category (c): Inter-cell communication timing non-determinism

What it is. The timing at which one cell's writes become visible to another cell varies. Cells communicate through substrate (A2.12, derived from §4.1), and reads return substrate's current state per Guarantee A. The latency between a writing cell's commit and a reading cell's read is determined by execution timing on both sides — the writer's commit time plus the reader's read time — and varies with infrastructure conditions.

Why it is operationally necessary. Deployments cannot synchronize inter-cell communication because cells run independently. Requiring synchronized communication would foreclose practical distributed deployment and would impose a synchronization layer that the architecture explicitly avoids.

How it is bounded. A2.12's no-direct-cell-to-cell-channels commitment (derived from §4.1) requires that cells communicate exclusively through substrate. Direct channels — message buses between cells, shared memory, RPC links — are not part of the architecture. Guarantee A (A2.57) commits to substrate state being read deterministically: at any moment, a read returns substrate's current content. The two commitments together bound inter-cell communication timing. The *moment* a cell reads substrate is non-deterministic per category (b); the *content* the read returns is deterministic per Guarantee A given substrate's state at read time. Communication timing varies; communication content does not.

5. Category (d): External-system response non-determinism

What it is. Adjacent components composed with substrate per A1.16's hybrid-systems patterns (RAG retrieval indexes, fine-tuned LLMs, vector databases, external structured stores, third-party APIs) may respond non-deterministically. Some external systems are eventually consistent; some have latency variation that affects response content; some return varying results based on internal state the architecture does not control.

Why it is operationally necessary. The architecture is technology-agnostic (A1.05); deployments compose with various external systems for various operational purposes. Requiring external-system determinism would constrain composition to a narrow set of components, would foreclose integration with existing enterprise infrastructure, and would substitute a specific technology choice for the architecture-agnostic posture the source paper commits to (§7.1, §7.4).

How it is bounded. A1.16 (derived from §2.3, §6.1, §6.2) defines three composition patterns for adjacent components: Pattern A (input to a cell), Pattern B (derivative view of substrate), Pattern C (separate concern). Adjacent components do not write to substrate authoritatively — they inform cell reasoning (Pattern A), provide derived views (Pattern B), or operate on concerns substrate does not own (Pattern C). External-system non-determinism may affect cell inputs in Pattern A or derived views in Pattern B, but cell writes to substrate remain governed by Property B's rule-structuring (per category (a)'s bounding), and Guarantee E (A2.61) commits substrate to be the source of truth. Where an external system's response varies, the cell consuming that response writes a rule-conformant record of what it received, not the external system's authority projected onto substrate.

6. Category (e): Hardware and infrastructure non-determinism

What it is. Underlying hardware, networking, and storage infrastructure produce non-deterministic effects: latency variation in storage backends, transient failures, resource contention, memory ordering across multi-core systems, network packet timing, CPU scheduling variation. Infrastructure produces these effects regardless of what runs on top of it.

Why it is operationally necessary. The architecture is technology-agnostic (A1.05); deployments operate across various hardware, network, and storage configurations. Requiring deterministic infrastructure would constrain the architecture to specific hardware platforms (and frequently to research-grade rather than production infrastructure), foreclosing practical deployment. The three minimal requirements (A2.24–A2.26) do not include infrastructure determinism for this reason.

How it is bounded. Tool-agnosticism (A1.05, derived from §7.1, §7.4) commits the architecture to operating compatibly with any infrastructure that satisfies the three minimal requirements — persistent structured state, human read/write access, and LLM access. The architectural-content guarantees hold at the substrate-content level regardless of the underlying infrastructure: Guarantee A (A2.57) over read determinism and Guarantee C (A2.59) over change addressability are commitments about substrate content, not about infrastructure timing. Implementation mechanisms that absorb infrastructure non-determinism — transactional storage, idempotent commit protocols, replication with consistency guarantees — operate beneath the architectural commitment. Guarantee E (A2.61) ensures that substrate's determinism is substrate-internal, not contingent on infrastructure determinism.

7. What allowed non-determinism does NOT include

The standalone treatment of allowed non-determinism is bounded as precisely as it is permissive. Stating what the five categories do *not* cover is what keeps the specification from drifting into a general

license.

Substrate read variation is not allowed. Reads against the same substrate state must return the same content per Guarantee A. A read that varies across operations against unchanged substrate is forbidden non-determinism, not category (b) timing variation.

Rule-bypass cell behavior is not allowed. Cell writes to substrate must be rule-conformant per Guarantee B. A cell whose behavior varies outside the bounds the rule allows is forbidden non-determinism, not category (a) LLM-output variation. Category (a) is bounded specifically by Property B's rule-structuring; the rule must hold.

Non-addressable changes are not allowed. Each substrate change must be addressable per Guarantee C. A partial commit, a change written without identifier, or a change whose record is lost between commit and read is forbidden non-determinism, not category (b) timing variation.

Conflict erasure is not allowed. Conflict states must be preserved deterministically per Guarantee D. A system whose handling of concurrent writes silently elides one of them is forbidden non-determinism, not category (c) communication-timing variation.

External-system overrides of substrate are not allowed. Substrate's source-of-truth status must hold per Guarantee E. A system in which an adjacent component can authoritatively rewrite substrate content is forbidden non-determinism, not category (d) external-response variation.

The five categories are also not adjacent patterns commonly conflated with them. Allowed non-determinism is not *unbounded* non-determinism — the categories are specifically bounded, and non-determinism outside them is forbidden by the guarantees. It is not *pseudo-deterministic execution* — deployments may strengthen determinism beyond the architectural commitment, but the architecture does not require pseudo-determinism. It is not *weak consistency* — weak consistency models that permit substrate reads to vary across replicas violate Guarantee A, regardless of which consistency vocabulary the implementation invokes. And it is not *free non-determinism for operational convenience* — the five categories are operationally necessary because current technology cannot avoid them; non-determinism introduced for convenience and not bounded by the named architectural mechanisms is forbidden.

The architecture also does not foreclose deployment-layer determinism strengthening. A deployment may configure its LLM for more deterministic output, may use deterministic schedulers, may select infrastructure with stronger consistency. Strengthening is a deployment choice; the architectural commitment is to allowed non-determinism not exceeding the five categories.

8. Failure modes

The following failure modes violate the boundaries between allowed and forbidden non-determinism. Each represents a misattribution that an implementation may make under operational pressure.

(a) **Treating substrate read variation as category (a) or (b) non-determinism.** Reads varying across operations against unchanged substrate violate Guarantee A; they are not allowed under any of the five categories.

(b) **Treating rule-bypass cell behavior as category (a) variation.** Category (a) is bounded by Property B's rule-structuring; cell behavior outside rule bounds violates Guarantee B and is forbidden, not permitted as LLM output variation.

(c) **Treating non-atomic commits as category (b) timing variation.** A2.10 requires atomic commits at boundary crossings; partial commits violate Guarantee C and are not category-(b) timing.

(d) **Treating external-system overrides as category (d) external-response variation.** Category (d) permits adjacent components to respond non-deterministically; it does not permit them to write to substrate authoritatively. Overrides violate Guarantee E.

(e) **Treating infrastructure non-determinism as license to violate substrate determinism.** Category (e) permits infrastructure variation; it does not permit substrate's content-level guarantees to fail. Implementation mechanisms must absorb infrastructure variation beneath the architectural commitment.

(f) **Conflating allowed non-determinism with weak consistency.** Weak consistency models that permit substrate reads to vary across replicas violate Guarantee A regardless of the consistency vocabulary invoked; they are not within any of the five categories.

(g) **Allowing LLM-output-as-substrate-write without rule structuring.** Treating category (a) as permission to write LLM outputs directly to substrate violates Property B and Guarantee B; rule-structuring is the mechanism by which category (a) is bounded.

(h) **Treating allowed non-determinism as license to introduce new sources of non-determinism.** The five categories are those that current technology requires; introducing additional architectural sources of non-determinism (random elements in rules, time-dependent rule logic, non-deterministic substrate reads chosen by design) is forbidden, not permitted under any category.

9. Operational test

A system satisfies the allowed-non-determinism boundaries if and only if all of the following hold.

1. All non-determinism in the system is attributable to one of the five categories named in §§2–6.
2. Each category's non-determinism is bounded by the specific architectural mechanism specified — Property B for (a), atomic boundary crossings for (b), no-direct-channels for (c), hybrid-systems composition patterns for (d), tool-agnosticism with substrate-internal determinism for (e).
3. Non-determinism does not propagate from any of the five categories to substrate's determinism properties per Guarantees A, B, C, D, E.
4. The system does not introduce architectural non-determinism outside the five categories.
5. The bounding mechanisms operate at the architectural-pattern level, not as deployment-toggleable features; deployments cannot disable the bounding while still claiming determinism-contract compliance.
6. Operational verification through regression testing (A2.63) confirms that substrate determinism holds across executions despite non-determinism in the allowed categories.

A system that fails any of (1)–(6) does not satisfy the allowed-non-determinism boundaries in the architectural sense, even if its operational behavior happens to be well-bounded in practice.

10. Conclusion

Allowed non-determinism in the CKS pattern is a positive architectural specification, not a residual category. The five categories — LLM token output, cell execution timing, inter-cell communication timing, external-system response, hardware and infrastructure variation — are the specific layers at which current technology cannot deliver determinism and the architecture operates compatibly with that fact. Each category is paired with a specific architectural mechanism that prevents non-determinism in the category from propagating to substrate. The pairing is what distinguishes the specification from a general license and what makes the determinism contract operational under current technology.

The standalone treatment matters because the typical implementation failure pattern is misattribution — non-determinism that should be forbidden is permitted under a misread category, or non-determinism that should be permitted is forbidden under a misread guarantee. Naming the five categories with their bounding mechanisms gives downstream readers the vocabulary to identify each direction of misattribution and the operational tools to test against it. The companion note A2.63 formalizes regression testing as the operational verification mechanism that, together with this specification, closes the determinism-contract decomposition.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Allowed Non-Determinism in the Coordination Knowledge Substrate Pattern: Standalone Treatment of the Five Categories the Determinism Contract Permits*. 5 May 2026. ORCID: 0009-0004-8065-3235.